RetailIQ: Full-Cycle E-Commerce Customer & Sales Analytics

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

M. Haswanthi	**: 2420030143**
Kunchala Naga Chaitanya Sphoorthy	**: 2420030510**
Kolapally Vasavi	**: 2420030785**

Under the Esteemed Guidance of

Ms. Prathusha

Assistant Professor

Department of Computer Science and Engineering

AY- 2026-2027

Semester: III



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University



DEPARTMENT OF CSE

Declaration

The Project Report entitled "**RetailIQ: Full-Cycle E-Commerce Customer & Sales Analytics**" is a record of Bonafide work of **M. Haswanthi – 2420030143, Kunchala Naga Chaitanya Sphoorthy – 2420030510, Kolapally Vasavi – 2420030785** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

M. Haswanthi – 2420030143

Kunchala Naga Chaitanya Sphoorthy – 2420030510

Kolapally Vasavi – 2420030785

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the project based report entitled "**RetailIQ: Full-Cycle E-Commerce Customer & Sales Analytics**" is a bonafide work done and submitted by **M. Haswanthi – 2420030143, Kunchala Naga Chaitanya Sphoorthy – 2420030510, Kolapally Vasavi – 2420030785** in partial fulfillment for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to our Course Coordinator **Nalajala Sunanda**, and our guide **Ms. Prathusha**, Assistant Professor, Department of Computer Science and Engineering, which cannot be expressed in words, for her tremendous support, encouragement and guidance for this project.

We express our gratitude to Dr. P. Venkateshwar, Head of the ***Department of Computer Science and Engineering***, for providing us with adequate facilities, ways and means by which we were able to complete this project.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

M. Haswanthi – 2420030143

Kunchala Naga Chaitanya Sphoorthy – 2420030510

Kolapally Vasavi – 2420030785

Abstract

E-commerce businesses record every purchase, but raw transaction logs do not by themselves tell a retailer who its best customers are, which customers are drifting away, which products and markets earn the most, or how revenue is likely to move next. RetailIQ is a full-cycle e-commerce analytics system that turns raw online-retail transaction data into this kind of business intelligence inside one easy-to-use web application.

The system accepts a CSV or Excel transaction file through a drag-and-drop upload page, normalises its column names, validates it, and cleans it. On the UCI Online Retail dataset, 541,909 raw records are reduced to 392,692 reliable records covering 4,338 customers, 3,665 products and 37 countries. Revenue and time features are derived, and each customer is scored on Recency, Frequency and Monetary value (RFM) and placed in one of six segments: Champions, Loyal, Regular, New, At Risk and Lost. In the sample data, the Champions segment is 22.1% of the customers but produces 65.2% of the revenue. A rule-based retention engine attaches a concrete action plan to every segment, and a Linear Regression model projects monthly revenue for one to six months ahead.

Results are presented in a six-page Streamlit dashboard (Overview, Sales Analytics, Customers, Products & Countries, Forecast and AI Assistant) with date and country filters, light and dark themes, a live date and time display and a fixed no-scroll layout. A conversational assistant answers questions about the data in a free rule-based mode or in a Gemini-based AI mode, which retries automatically and falls back to the rule-based answer when the language-model service is busy. The complete system runs natively in Python or in a Docker container, giving small retailers a low-cost, practical decision-support tool.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	ii
		Abstract	iii
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	1-4
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	5-6
3	Technology Stack	3.1 Front-end (Streamlit, Plotly, HTML/CSS) 3.2 Back-end (Python, Pandas, scikit-learn) 3.3 Database (file-based data layer) 3.4 Version Control (Git) 3.5 APIs or External services (Gemini API)	7-8
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer 4.3 Deployment architecture	9-12
5	Design	5.1 Wireframes or mockups 5.2 ER Diagram / Database schema	13-16
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (UI rendering, state management) 6.3 Backend logic (cleaning, segmentation, forecasting) 6.4 Database connectivity (data loading and caching) 6.5 API integration (Gemini assistant) 6.6 Authentication & Authorization	17-22
7	Features	7.1 List of main features 7.2 How each feature works (user perspective + technical description)	23-29
8	Testing	8.1 API and backend testing (Postman) 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	30-32
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	33-34
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	35-36
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	37
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	38-39
13	References	- Books, tutorials, APIs, documentation sites used	40-41
14	Appendices	A. Screenshots of the app B. Sample code snippets C. Installation/setup instructions D. User manual or guide E. Certifications	42-49

CHAPTER 1

Introduction

1.1 Background of the project

Electronic commerce has become one of the main ways in which goods are bought and sold. Every order placed on an online store leaves a record: which invoice it belonged to, which product was bought, in what quantity, at what price, on which date and hour, by which customer and from which country. A medium-sized online retailer therefore accumulates hundreds of thousands of such lines within a year. These records are valuable, but in their raw form they answer none of the questions a business owner actually asks: Who are my most valuable customers? Who has stopped buying? Which products and countries earn the most? How much will I sell next month?

Traditional spreadsheets can total a column or draw a simple chart, but they become slow and error-prone as data grows, and they offer no systematic way to group customers by behaviour or to project future revenue. Large enterprises solve this with dedicated data-warehouse and business-intelligence platforms, which are expensive and need specialist staff. Small and medium retailers need something lighter: a tool that can be given a transaction file and returns cleaned data, customer segments, product and market rankings, a forecast and clear recommendations.

RetailIQ (Full-Cycle E-Commerce Customer & Sales Analytics) is built to be that tool. It combines data cleaning, feature engineering, exploratory sales analysis, RFM-based customer segmentation, a rule-based retention recommendation engine, linear-regression revenue forecasting, an interactive dashboard and a conversational AI assistant in one Python application. The system is demonstrated on the well-known UCI **Online Retail** dataset [4], which contains 541,909 transaction lines from a UK-based online gift-ware retailer between 1 December 2010 and 9 December 2011, and it also accepts the newer **Online Retail II** layout [5].

1.2 Problem statement

Raw e-commerce transaction files are large, noisy and unlabelled. They contain duplicate rows, cancelled orders with negative quantities, missing customer identifiers and invalid prices, and

different releases of the same data use different column names. Even after cleaning, a plain table of transactions does not tell a business which customers deserve rewards, which need win-back offers, or whether revenue is rising.

The problem addressed by this project is therefore: how can raw online-retail transaction data be converted, automatically and in one place, into reliable customer, product, market and revenue insights, together with actionable retention advice, for users who are not data scientists?

1.3 Scope of the project

Within the scope of the project:

1. Uploading CSV and XLSX online-retail transaction files, or using the bundled sample dataset, and validating that the required columns are present.
2. Cleaning the data and deriving revenue and time features (year, month, day, hour).
3. Sales analysis: key performance indicators, monthly trend, orders by weekday and by hour, product and country rankings.
4. Customer analysis: RFM scoring, six behavioural segments, customer-level indicators and segment-wise retention recommendations.
5. Revenue forecasting for one to six months ahead using Linear Regression, and an offline backtest of its accuracy.
6. An interactive six-page dashboard with filters, light and dark themes, and a rule-based and Gemini-based assistant.
7. Packaging with Docker and Docker Compose so the application runs identically on any machine.

Outside the scope of the project: processing live transactions, connecting to payment, inventory or order-management systems, user accounts and multi-user access control, real-time currency conversion, and advanced forecasting models with seasonality. These are listed as future work in Chapter 11.

1.4 Objective(s)

1. To build a reliable pipeline that normalises, validates and cleans raw e-commerce transaction data and derives revenue and time features.

2. To segment customers into six interpretable behavioural groups using Recency, Frequency and Monetary (RFM) analysis.
3. To connect every customer segment with a concrete retention or win-back action through a rule-based recommendation engine.
4. To project future monthly revenue with an interpretable regression model and to measure its accuracy honestly against simple baselines.
5. To present all results in an interactive dashboard that starts from a simple drag-and-drop upload and works in light and dark themes on one screen without scrolling.
6. To provide a natural-language assistant that answers questions about the analysed data, and a portable Docker-based deployment.

1.5 Importance of the application

Customer value is rarely spread evenly. In the sample dataset the average customer spent about £2,049, but the median customer spent only about £669, and the 957 customers in the Champions segment (22.1% of all customers) produced 65.2% of the total revenue. A retailer that cannot see this concentration cannot protect its best customers or decide where retention budget should go. At the other end, 1,065 customers (24.6%) are already Lost and a further 643 (14.8%) are At Risk; these are the customers for whom a timely win-back offer is still worth sending.

RetailIQ makes this visible within seconds of uploading a file. It also removes the technical barrier: no code, database or paid business-intelligence licence is needed. The upload-first design, instant validation messages, one-click navigation and consistent visual theme are intended to make an analytics workflow feel like a finished product rather than a notebook.

1.6 Target users/audience

7. **Small and medium online retailers** who want to understand their customers and products without hiring a data team.
8. **Marketing and CRM teams** who need ready-made segments and campaign ideas for loyalty, onboarding and win-back.
9. **Business analysts and students** who want a transparent, reproducible example of an end-to-end retail-analytics pipeline.

10. **Instructors and evaluators**, for whom the sample dataset and the Docker image allow the project to be demonstrated on any computer.

CHAPTER 2

System Requirements

2.1 Hardware requirements

RetailIQ is a lightweight application. The main resource demand comes from reading the 541,909-row Excel workbook the first time; after that the cleaned data occupies about 100 MB of memory and every analysis result is cached. In our test environment the first read of the Excel file took tens of seconds to about a minute, while loading the bundled cleaned CSV and all later page changes took only a few seconds.

Table 2.1 Hardware requirements

Component	Minimum	Recommended
Processor	Dual-core, 2 GHz	Quad-core (or better), 2.5 GHz
Memory (RAM)	4 GB	8 GB or more
Storage	2 GB free (project, Python packages)	5 GB free (also holds the Docker image)
Display	1366 x 768	1920 x 1080 (the fixed layout adapts to the window height)
Network	Not needed for rule-based mode	Internet access for Gemini AI mode and the Google Fonts web font
GPU	Not required	Not required

2.2 Software requirements

Table 2.2 Software requirements

Software	Requirement / version
Operating system	Windows 10/11, macOS or Linux
Language	Python 3.11 (the version used in the Docker image); any recent Python 3.x that supports the libraries below
Web browser	A current Chrome, Edge, Firefox or Safari
Container platform (optional)	Docker Engine or Docker Desktop with Docker Compose
Python libraries	pandas, numpy, scikit-learn, streamlit (version 1.50 or newer), plotly, openpyxl, google-genai, python-dotenv (all listed in requirements.txt)
Versions used for testing	Streamlit 1.65, Plotly 7.1, pandas 3.0, scikit-learn 1.8, openpyxl 3.1

2.3 Development tools and frameworks

11. **Python and Jupyter Notebook** for the exploratory work. The project contains six notebooks: data understanding, customer analysis, product and sales analysis, customer segmentation, sales forecasting and dashboard preparation.
12. **Streamlit** as the application framework, which turns Python scripts into an interactive web application without separate front-end code.
13. **Plotly** for interactive charts, and **Matplotlib and Seaborn** for the static charts used in the notebooks.
14. **Git** for version control, and **Docker / Docker Compose** for packaging and deployment.
15. **Streamlit AppTest and a headless Chromium browser** for automated testing of the interface (Chapter 8).

CHAPTER 3

Technology Stack

3.1 Front-end (Streamlit, Plotly, HTML/CSS)

The front end is written entirely in Python with **Streamlit** [6]. Streamlit provides the page layout (columns, containers, tabs), widgets (file uploader, radio navigation, date input, multiselect, toggle, slider, buttons, chat input) and the rerun-on-interaction model. Charts are drawn with **Plotly** [11], which gives hover tooltips and responsive sizing.

A custom stylesheet, injected as HTML/CSS, gives the application its own look: a soft pink and purple palette, gradient buttons and cards, rounded corners and the Inter web font. All colours are CSS variables, so the whole interface switches between a light and a dark theme instantly. A few lines of JavaScript run the live clock in the top bar. No separate front-end framework such as React or Angular is used.

3.2 Back-end (Python, Pandas, scikit-learn)

The back end is a set of plain Python modules in the `src` folder. **Pandas** [9] and **NumPy** [10] handle loading, cleaning, grouping and feature engineering. **scikit-learn** [2] provides the LinearRegression model used for forecasting. There is no separate server process or REST API: the Streamlit server calls these modules directly, which keeps the system simple to run and to test.

3.3 Database (file-based data layer)

RetailIQ does not use a database management system. Input data arrive as CSV or Excel files (read with pandas and openpyxl), are held in memory as DataFrames, and results are kept in Streamlit's in-memory cache. This is a deliberate choice: the application analyses a dataset that the user supplies, it stores nothing permanently, and it therefore needs no database installation. The bundled sample is the pre-cleaned file `data/processed/clean_sales_data.csv`, and the raw workbook is `data/raw/Online Retail.xlsx`. The logical data model is described in Section 5.2.

3.4 Version Control (Git)

The project folder is a Git repository. The `.gitignore` and `.dockerignore` files exclude the virtual environment, caches, secrets (`.env`, `secrets.toml`) and archives, so that API keys are never committed or copied into the container image.

3.5 APIs or External services (Gemini API)

The only external service is the **Gemini API** [8], accessed through the Google GenAI Python SDK, and it is optional. It powers the open-ended AI mode of the assistant. The application sends the model a compact text summary of the analysed data rather than the raw file, and works fully without it. The Google Fonts service is used only to load the Inter typeface for the interface.

Table 3.1 Summary of the technology stack

Layer	Technology	Purpose
Interface	Streamlit, Plotly, HTML/CSS, small JavaScript	Pages, widgets, charts, theming, live clock
Data processing	Python, Pandas, NumPy, openpyxl	Loading, normalising, cleaning, aggregating
Machine learning	scikit-learn (LinearRegression)	Monthly revenue forecast
Rules engine	Plain Python	RFM segment rules, retention playbook, rule-based chatbot
Generative AI	google-genai SDK, Gemini API	Open-ended question answering
Packaging	Docker, Docker Compose, python-dotenv	Portable deployment, key configuration

CHAPTER 4

System Architecture

4.1 High-level architecture diagram

RetailIQ follows a five-layer architecture: ingestion, processing, intelligence, presentation and deployment. Data flows downward from one layer to the next, and each layer depends only on the layer above it, so a layer can be changed without disturbing the others. For example, the chart styling can be redesigned without touching the analytics code, and a different forecasting model can be inserted without changing the dashboard pages.

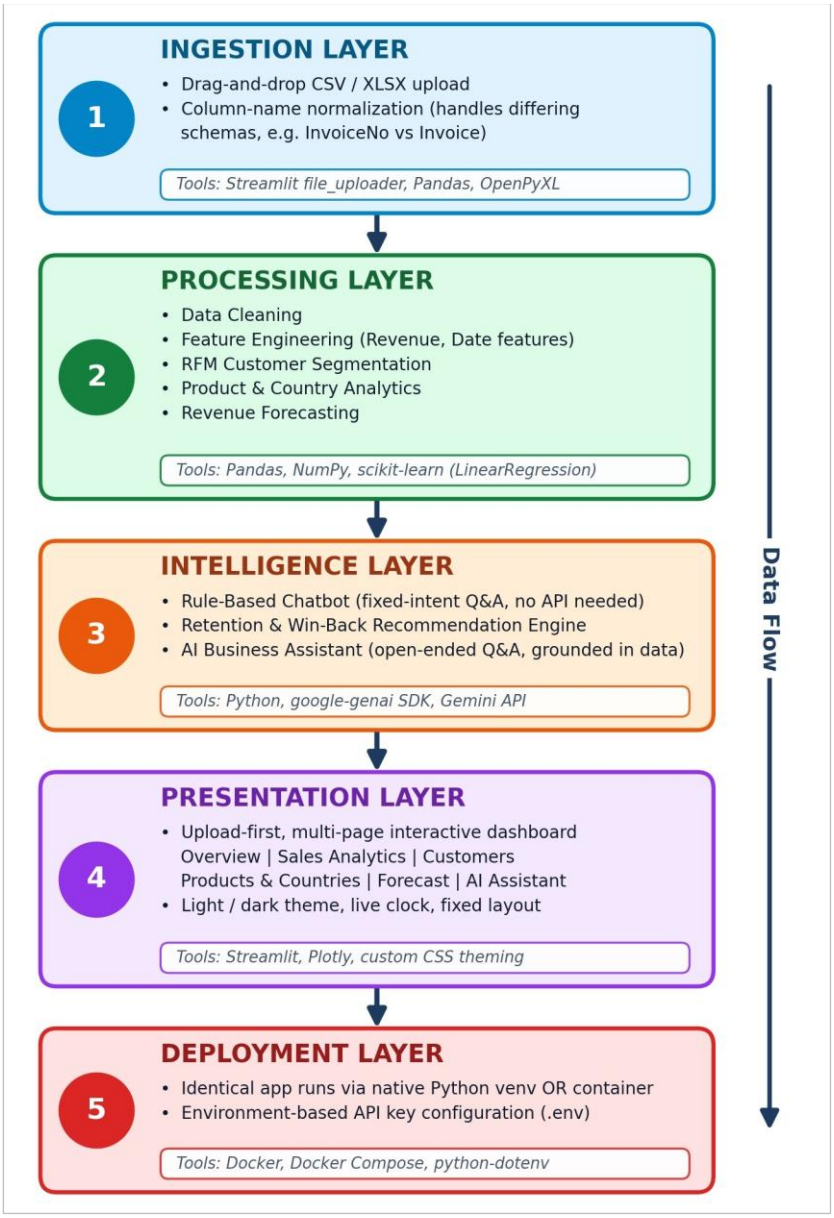


Fig No. 4.1 RetailIQ five-layer system architecture

4.2 Description of each layer

Layer 1: Ingestion layer (back end)

The user drops a CSV or XLSX file on the upload page. The file is read with pandas (openpyxl for Excel). A column alias map renames the headers used by different dataset releases (for example Invoice, Price, Customer ID of Online Retail II) to the standard schema InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, Country. If any required column is missing, the file is rejected with a message that names the missing columns.

Layer 2: Processing layer (back end)

The data are cleaned (duplicates, missing customer identifiers, cancelled or negative-quantity lines, non-positive prices), revenue and date features are derived, and the analytical aggregates are computed: RFM table and segments, product performance, country performance and monthly sales with the forecasting model.

Layer 3: Intelligence layer (back end)

This layer turns numbers into advice and answers. The retention engine maps each segment to a headline and a list of actions. The rule-based chatbot answers fixed-intent questions directly from the computed tables. The AI business assistant calls the Gemini model with a summary of the analysis and the recent conversation.

Layer 4: Presentation layer (front end)

A six-page Streamlit dashboard shows the results, with sidebar navigation, date and country filters, a light/dark switch and a live clock. Every page recomputes from the filtered data.

Layer 5: Deployment layer

The same application can be started from a native Python virtual environment or from a Docker container. Configuration, notably the optional Gemini API key, is read from environment variables or a `.env` file.

4.3 Deployment architecture

The application is deployed as a single container (Fig. 4.2). The Streamlit server and the analytics modules run in the same Python process inside a `python:3.11-slim` image and are exposed on port 8501. A Docker health check calls Streamlit's `/_stcore/health` endpoint. Docker Compose mounts the host `data` folder into the container so that new datasets

can be added without rebuilding the image, passes `GEMINI_API_KEY` from the host environment, and restarts the container automatically. There is no CI/CD pipeline or cloud hosting in the current version; the container can be started on any machine that has Docker, and it can be moved to a cloud service later (Chapter 9).

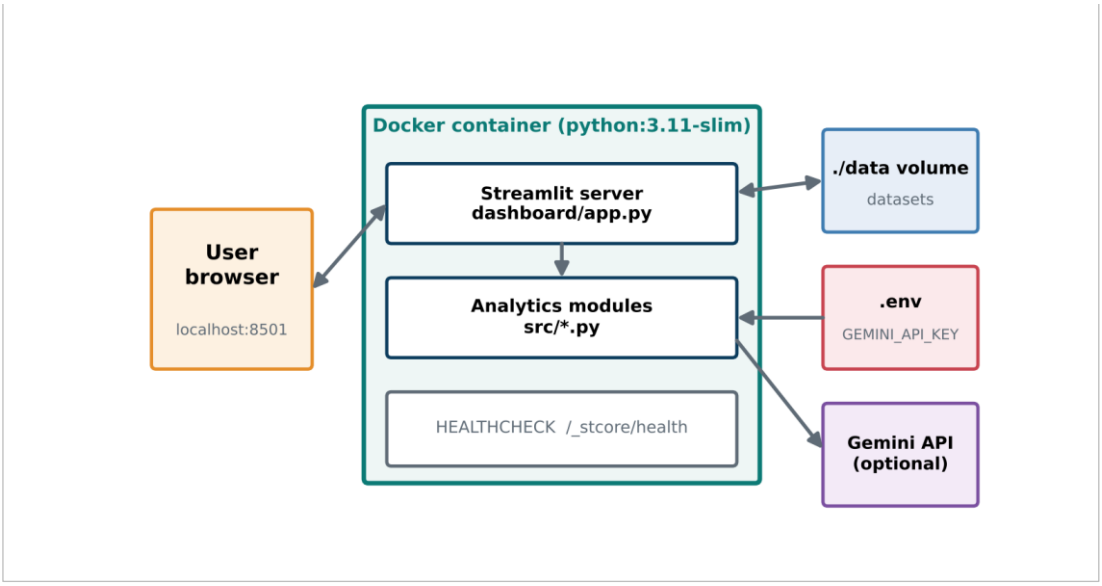


Fig No. 4.2 Deployment architecture of RetailIQ (Docker)

CHAPTER 5

Design

5.1 Wireframes or mockups

The interface was designed around three ideas. First, **upload first**: the user sees only a large drag-and-drop box and a sample-dataset button, and the dashboard appears after a file has been accepted. Second, **one screen per page**: every page fits the browser window without page scrolling, so charts and tables resize to the available height. Third, **consistent, calm visuals**: a soft pink and purple palette, rounded cards, one navigation pattern and a light and a dark theme.

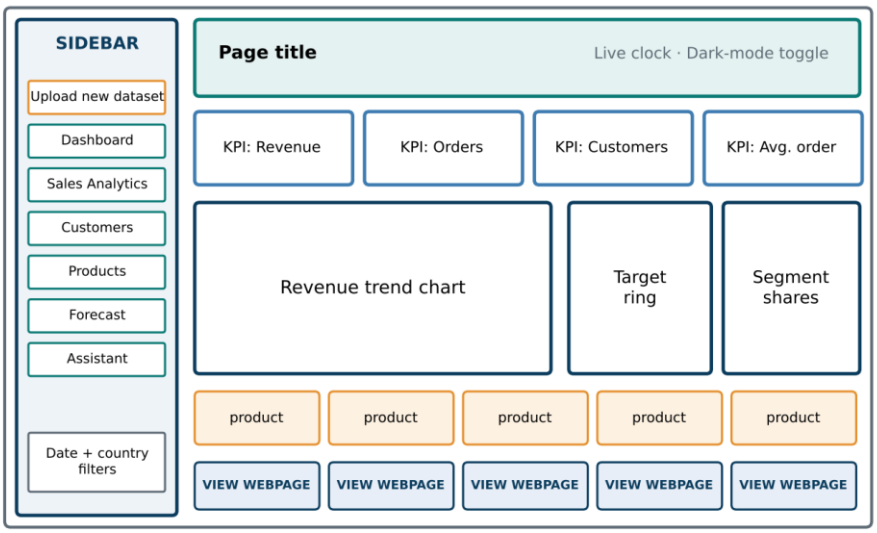


Fig No. 5.1 Wireframe of the Overview page (sidebar, top bar, KPI cards, charts, product strip and module cards)

The wireframe shows the Overview layout: a sidebar with navigation and filters, a top bar with the page title, live clock and theme switch, a row of four KPI cards, the revenue trend with the sales-target ring and customer-segment shares, the top-selling products strip, and a row of module cards each with a colourful **View Webpage** button. The finished interface for the first screen the user sees is shown below.

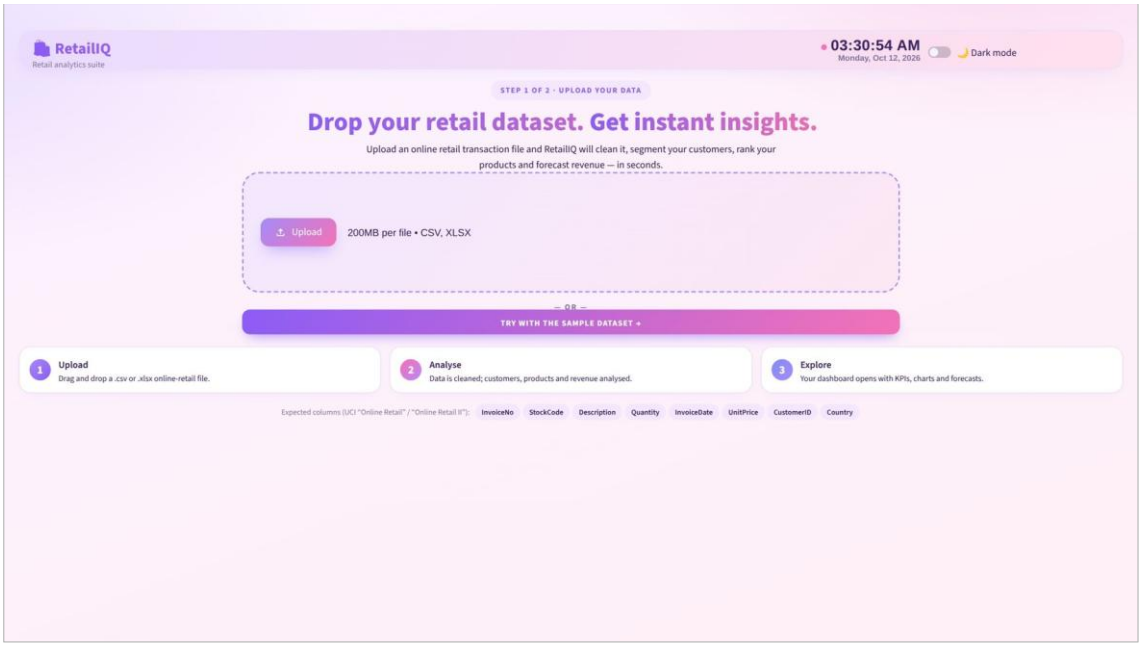


Fig No. 5.2 Upload page: drag-and-drop area, sample-dataset button and three-step guide

5.2 ER Diagram / Database schema

RetailIQ has no relational database, but it has a clear logical data model (Fig. 5.3). The cleaned transaction table is the single source of truth. Every other table is derived from it by grouping: the customer RFM table by `CustomerID`, the product summary by `StockCode`, the country summary by `Country`, and the monthly sales table by month. The segment playbook is a static lookup from segment name to recommendation.

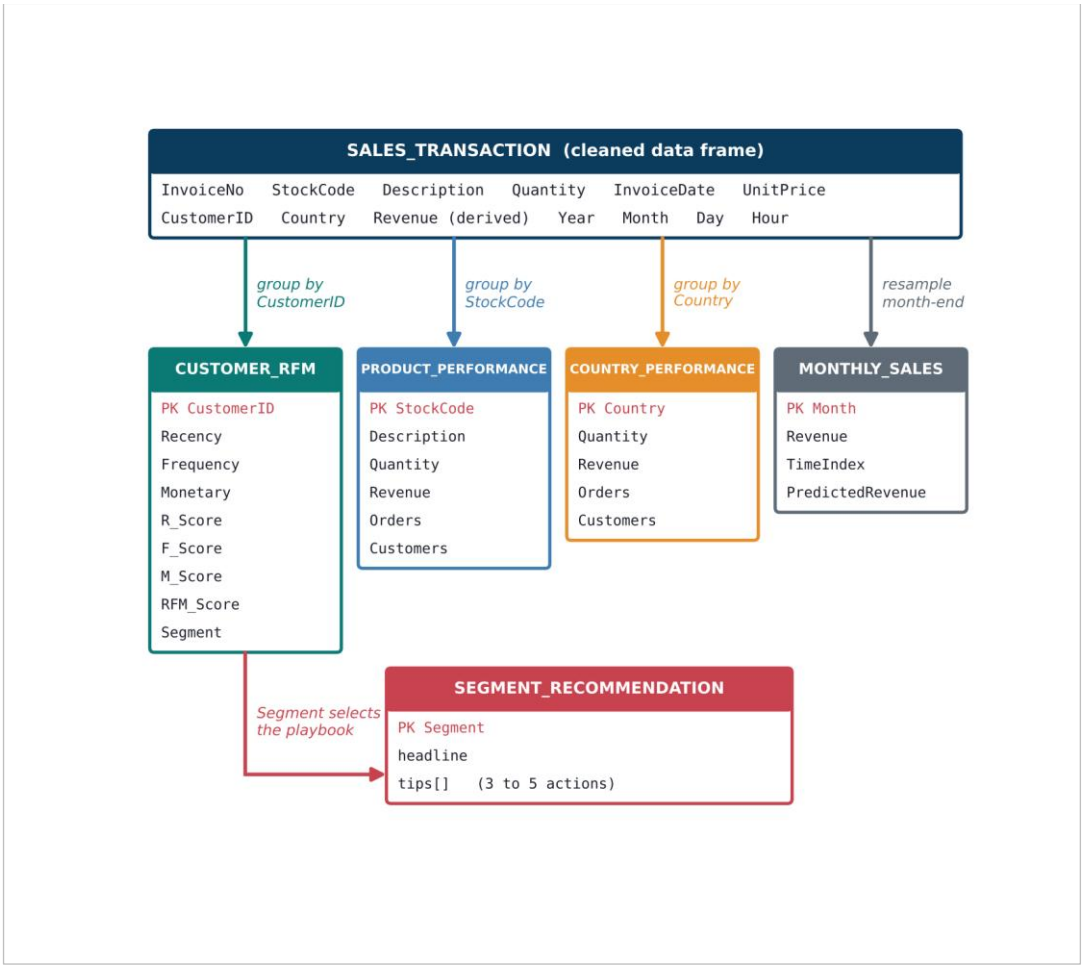


Fig No. 5.3 Logical data model of RetailIQ (entities and derivations)

Table 5.1 Data dictionary of the cleaned transaction table

Attribute	Type	Description
InvoiceNo	text	Invoice (order) identifier; one invoice has many lines
StockCode	text	Product code
Description	text	Product name
Quantity	integer	Units on the line (positive after cleaning)
InvoiceDate	datetime	Date and time of the invoice
UnitPrice	decimal	Price of one unit in pounds sterling (positive after cleaning)
CustomerID	number	Customer identifier (never missing after cleaning)
Country	text	Country of the customer
Revenue	decimal (derived)	Quantity x UnitPrice
Year, Month, Day, Hour	integer (derived)	Parts of InvoiceDate used for trends and hourly charts

Table 5.2 Derived tables

Table	Key	Columns
CUSTOMER_RFM	CustomerID	Recency (days), Frequency (distinct invoices), Monetary (total revenue), R/F/M scores 1-5, RFM_Score, Segment

Table	Key	Columns
PRODUCT_SUMMARY	StockCode	Description, Quantity, Revenue, Orders, Customers
COUNTRY_SUMMARY	Country	Quantity, Revenue, Orders, Customers
MONTHLY_SALES	Month	Revenue, TimeIndex, PredictedRevenue
SEGMENT_PLAYBOOK	Segment	Headline, list of recommended actions

CHAPTER 6

Implementation

6.1 Module-wise implementation

The code is split into small modules, each with one responsibility, so that every part can be tested on its own and reused outside the dashboard. The end-to-end processing flow is shown in Fig. 6.1.

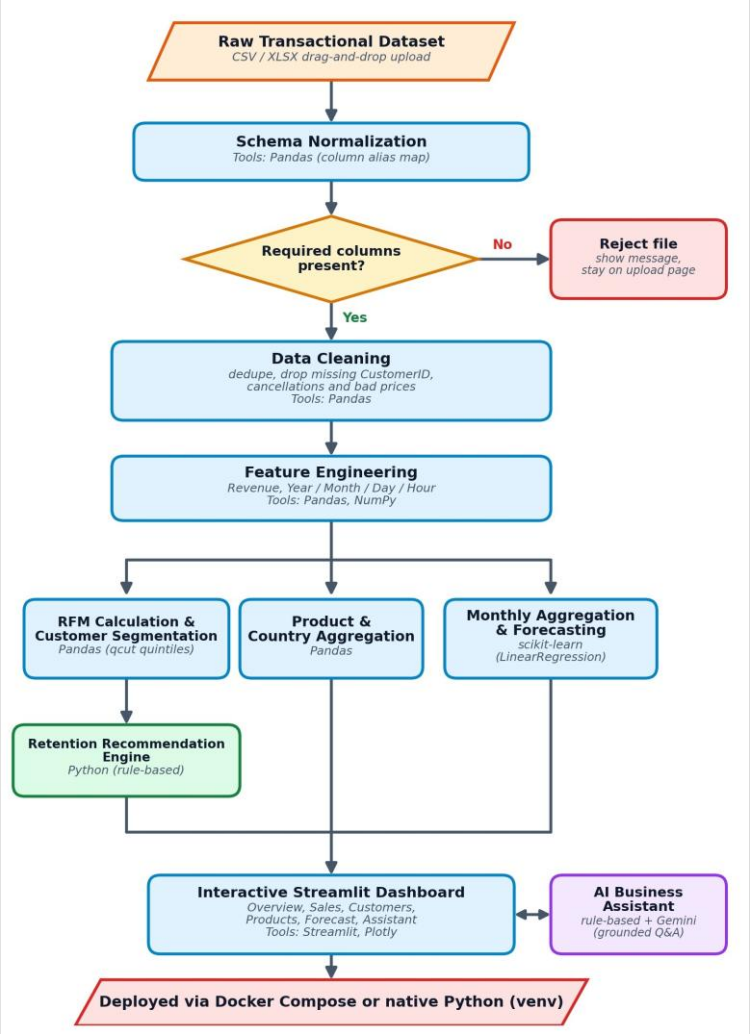


Fig No. 6.1 End-to-end processing flowchart of RetailIQ

Table 6.1 Modules and their responsibilities

Module (file)	Responsibility	Key functions
data_cleaning.py	Schema normalisation, validation, loading, cleaning	normalize_columns, load_data, clean_data
feature_engineering.py	Revenue and date features	create_revenue, create_date_features

Module (file)	Responsibility	Key functions
customer_analysis.py	RFM table and customer indicators	calculate_rfm, calculate_customer_kpis
segmentation.py	Quintile scores and six segments	calculate_rfm_scores, assign_customer_segments
retention.py	Retention and win-back playbook	get_recommendations, get_segment_summary
product_analysis.py	Product and country performance	calculate_product_performance, calculate_country_performance
forecasting.py	Monthly sales and Linear Regression	create_monthly_sales, train_forecasting_model, forecast_future
currency.py	Country to display-currency mapping	get_currency_for_countries
chatbot.py	Rule-based assistant	build_context, answer, set_currency
ai_assistant.py	Gemini-based assistant with retry and fallback	ask_ai, build_ai_context, get_api_key
product_icons.py, product_images.py	Icons and photos for product cards	get_icon_for_description, get_image_b64_for_description
dashboard/app.py	Streamlit application: upload flow, pages, filters	load_data (cached), page sections
dashboard/theme.py	Theme: CSS, light/dark variables, component HTML	build_css, kpi_card, feature_card

6.2 Front-end logic (UI rendering, state management)

Upload-first flow. The application keeps one value in Streamlit's session state, `dataset`. While it is empty, only the upload page is rendered and the script stops. When the user uploads a file, it is validated with the same loading function the dashboard uses; on success the file bytes and name are stored in `dataset` and the page reruns into the dashboard. The sample-dataset button stores an empty marker so that the bundled file is used. The sidebar button **Upload new dataset** clears the value and returns to the upload page.

Navigation and state. The six pages are selected by a radio widget whose state is held in `nav_page`. The **View Webpage** buttons on the Overview page use a callback that changes `nav_page`, so one click opens the corresponding page. The dark-mode switch is a keyed toggle, `dark_mode`; the stylesheet is generated from this value on every run.

Rendering and theming. Page content is built from Streamlit containers, columns and tabs. KPI cards, product cards, progress bars and module cards are small HTML snippets generated by functions in `theme.py`. All colours are CSS variables defined for the light and dark themes, and a shared Plotly template and explicit figure colours make the charts follow the theme.

Fixed no-scroll layout. Chart and table heights are not fixed in pixels; CSS rules size them from the window height (for example, the main chart gets the height left after the fixed parts of the page). Plotly resizes to its container when the figure has no fixed height, and a small script asks it to redraw after rendering. The result is that each page fits one screen at common sizes.

Live date and time. A short JavaScript timer in the top bar reads the browser clock every second and writes the time and date into the page, so no server rerun is needed.

6.3 Backend logic (cleaning, segmentation, forecasting)

Data cleaning

The cleaning function applies four rules in order (Table 6.2). The effect of each rule on the sample dataset was measured on the raw workbook.

Table 6.2 Effect of each cleaning rule on the Online Retail data

Step	Records removed	Records remaining
Raw dataset	-	541,909
Remove duplicate rows	5,268	536,641
Remove rows without CustomerID	135,037	401,604
Remove cancelled / negative-quantity lines	8,872	392,732
Remove zero or negative unit price	40	392,692

Revenue is then computed as Quantity x UnitPrice, and the year, month, day and hour are extracted from the invoice date.

RFM scoring and segmentation

The reference date is the day after the last transaction in the data. For every customer, Recency is the number of days since the last purchase, Frequency is the number of distinct invoices, and Monetary is the total revenue. Each measure is converted into a score from 1 to 5 using quintiles. Recency is scored in reverse (more recent is better) and frequency and monetary values are ranked first so that tied values can still be split into five groups. Segments are then assigned by the following rules, applied in this order:

Table 6.3 Segment rules (r, f, m are the R, F and M scores)

Order	Segment	Rule
1	Champions	r >= 4 and f >= 4 and m >= 4
2	Loyal Customers	r >= 3 and f >= 3 and m >= 3
3	New Customers	r >= 4 and f <= 2

Order	Segment	Rule
4	At Risk	r <= 2 and f >= 3
5	Lost Customers	r <= 2 and f <= 2
6	Regular Customers	all remaining customers

Retention recommendation engine

The retention engine is a dictionary from segment to a headline and a list of three to five actions, for example early access and loyalty perks for Champions, cross-sell suggestions for Loyal customers, a welcome series and second-purchase incentive for New customers, a time-limited win-back offer for At Risk customers, and a stronger incentive plus a short survey for Lost customers, with advice to stop after two or three unanswered attempts. For the segment chosen on the dashboard, it also reports the number of customers, the historical revenue at stake and the average days since the last order.

Revenue forecasting

Monthly revenue is obtained by summing revenue at month ends. A Linear Regression model is fitted on a month index t (0, 1, 2, ...) and the revenue of that month, and is extended to the next one to six months chosen by the user. For the sample data the fitted line has a slope of about £34,737 per month and an intercept of about £475,209, which gives forecasts of £926,791, £961,528 and £996,265 for January to March 2012.

6.4 Database connectivity (data loading and caching)

Data access is file based. The loading function reads CSV with `read_csv` and Excel with `read_excel`, normalises the columns, and, for uploaded files, runs the cleaning function. It is wrapped in Streamlit's `cache_data`, keyed by the file bytes and name, so that the 541,909-row workbook is processed only once per session. The analytical functions (monthly sales, RFM and segments, product and country tables) are cached on the filtered DataFrame, so they are recomputed only when a filter changes. When exactly one country is selected, revenue is multiplied once by an indicative exchange rate from the currency module so that every downstream number is shown in that country's currency; otherwise sterling is used.

6.5 API integration (Gemini assistant)

In AI mode the application builds a text summary of the analysis (KPIs, segment sizes and revenue, top products, countries and monthly sales) and the last six chat turns, and sends it with

a system instruction to the Gemini model through the Google GenAI SDK. The model is asked to answer only from the supplied data. The API key is read from the environment, a `.env` file, or a password box in the sidebar for the current session.

Hosted language-model services are sometimes overloaded; during development the service returned an HTTP 503 error with the message that the model was experiencing high demand. The integration therefore handles temporary errors explicitly:

16. Errors that normally clear by themselves (503 unavailable or high demand, 429 rate limit, 500/504 server errors, time-outs) are retried up to three times with a short back-off (1.5 s and then 3 s).
17. If the main model (`gemini-flash-latest`) stays busy, the lighter `gemini-flash-lite-latest` and then `gemini-2.5-flash` are tried in turn.
18. A rejected or missing API key is reported at once instead of being retried.
19. If every attempt fails, the user sees a clear message that the service is busy, and the dashboard answers the same question with the free rule-based assistant, so the user always receives an answer.

6.6 Authentication & Authorization

RetailIQ is a single-user analytics tool that runs on the user's own machine or container, so it has no login or role system. Access control is therefore the responsibility of the environment in which it is deployed (for example, a private network or a reverse proxy with authentication if it is ever hosted publicly). The security measures that are implemented are:

20. The Gemini API key is never written to disk by the application; it is read from an environment variable or `.env` file, or typed into a password field and kept only for the session.
21. `.env` and `secrets.toml` are listed in `.gitignore` and `.dockerignore`, so the key is not committed to Git or copied into the image.
22. Uploaded files are processed in memory and are not stored on the server.
23. Uploads are limited to `.csv` and `.xlsx` and are validated before analysis; malformed files produce an error message instead of a crash.
24. In AI mode only a summary of the analysis, not the uploaded file, is sent to the external service.

CHAPTER 7

Features

7.1 List of main features

Table 7.1 Main features of RetailIQ

No.	Feature	Short description
1	Upload-first start page	Drag-and-drop CSV/XLSX upload, sample dataset, validation messages
2	Automatic data preparation	Schema normalisation, cleaning, revenue and time features
3	Overview dashboard	KPI cards, revenue trend, sales-target ring, segment shares, top products, module cards
4	Sales analytics	Monthly revenue, orders by weekday, sales by hour of day
5	Customer analytics	RFM segmentation into six groups, customer KPIs, RFM table
6	Retention recommendations	Segment-specific headline and actions, revenue at stake
7	Products and countries	Product gallery, top-10 products, country revenue chart and table
8	Revenue forecast	Linear-regression forecast for 1-6 months with table and cards
9	Rule-based assistant	Free answers to fixed-intent questions about the data
10	Gemini AI assistant	Open-ended questions, automatic retry and fallback
11	Filters and currency	Date range and country filters; country currency display
12	Themes and live clock	Light / dark mode, live date and time, fixed no-scroll layout
13	Portable deployment	Docker and Docker Compose, single-command start

7.2 How each feature works (user perspective + technical description)

Upload page and data validation

User view: the user sees a large dashed box and drops a CSV or Excel file on it, or clicks **Try with the sample dataset**. A progress panel reports reading, cleaning and the number of clean transactions, customers and countries, and the dashboard then opens. A file with wrong columns is refused with a message that lists the missing columns. *Technical view:* the file bytes are passed to the cached loading function, which normalises the headers through the alias map, checks the required columns, cleans the data and returns a DataFrame; errors are caught and shown without leaving the upload page.

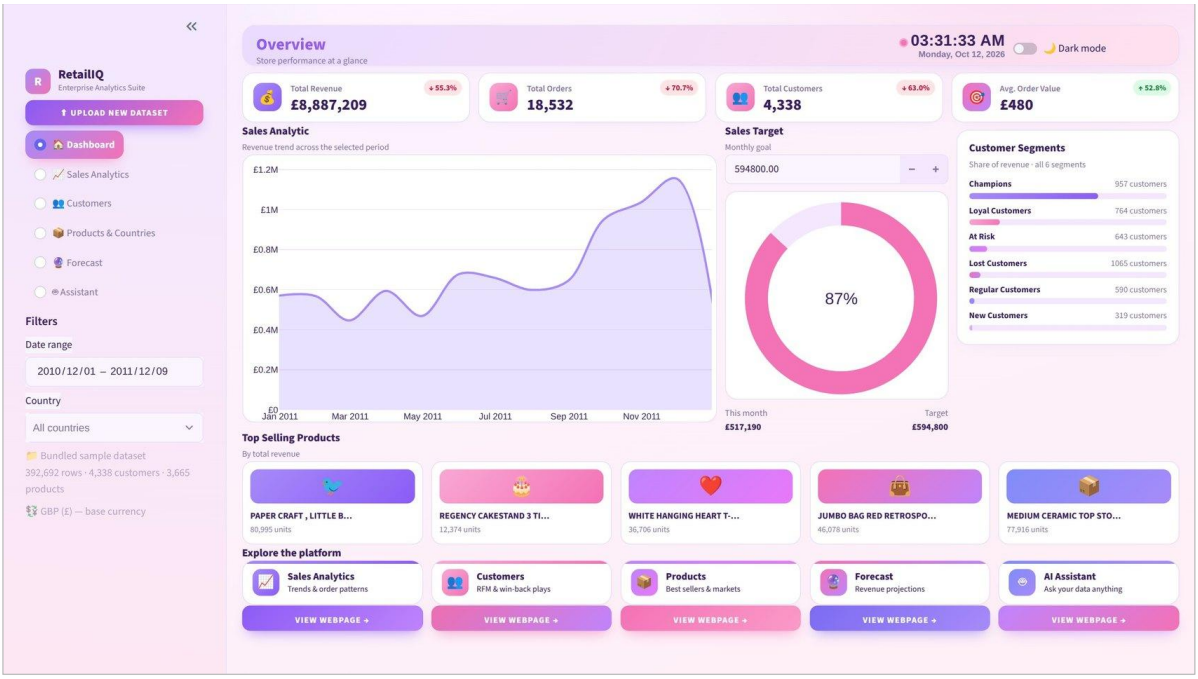


Fig No. 7.1 Overview page: KPI cards, revenue trend, target ring, segment shares, top products and View Webpage buttons

Overview dashboard

User view: the Overview page summarises the business in one screen: total revenue (£8,887,209), orders (18,532), customers (4,338) and average order value (£480), each with a badge comparing the last month with the previous one. The sales-target ring compares this month's revenue with an editable monthly target, the segment card shows the revenue share of all six segments, and the product strip shows the five best sellers. Each module card has a **View Webpage** button that opens that page. *Technical view:* the KPI values are sums and distinct counts over the filtered data; the month-over-month badge compares the last two months present. Because the data end on 9 December 2011, the final month is incomplete, so its badge shows a large apparent fall; this is a data limitation, not an error.

Sales analytics

User view: the page shows monthly revenue as an area chart, orders per weekday as bars and orders per hour of day. In the sample data Thursday has the most orders, no orders occur on Saturday, and order volume peaks around midday. *Technical view:* orders are counted as distinct invoices after grouping by weekday name or by the hour feature.

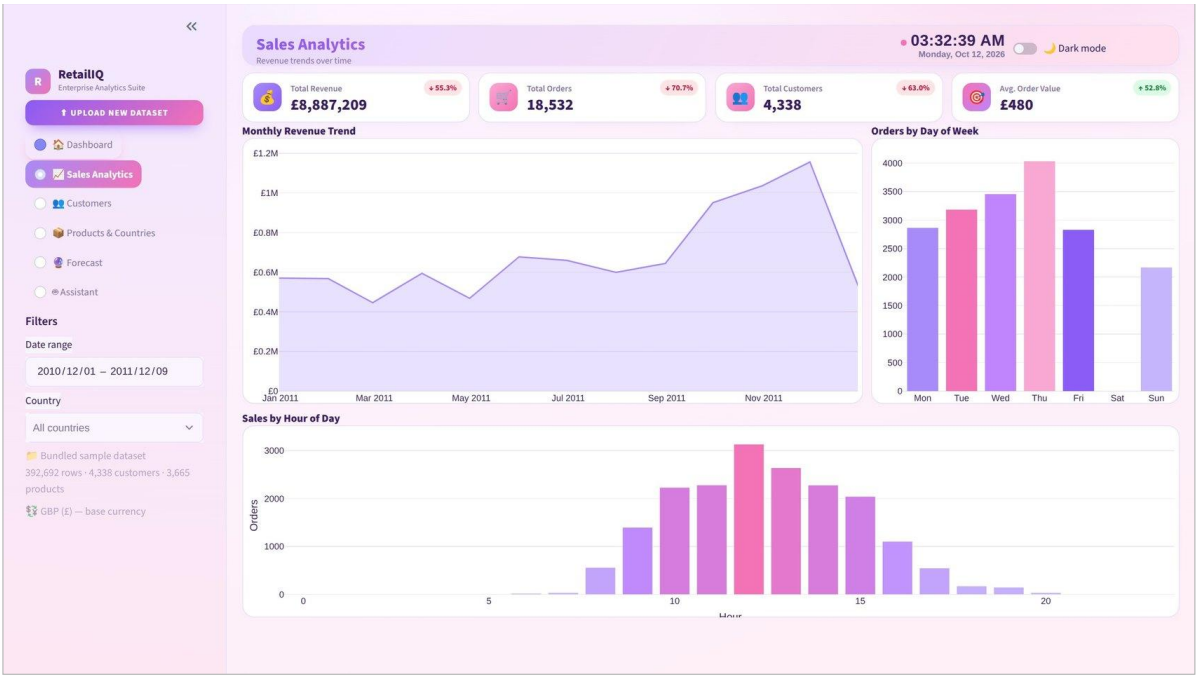


Fig No. 7.2 Sales Analytics page

Customer analytics and retention recommendations

User view: the Customers page shows customer-level indicators (average revenue £2,049, median £669, 4.27 orders per customer), a segment pie chart, a table of the highest-value customers with their RFM values, and a second tab where the user picks a segment and reads the recommended actions together with the number of customers, revenue at stake and average days since the last order. *Technical view:* the RFM table is computed from the filtered data, scored and segmented with the rules of Table 6.3, and passed to the retention module.

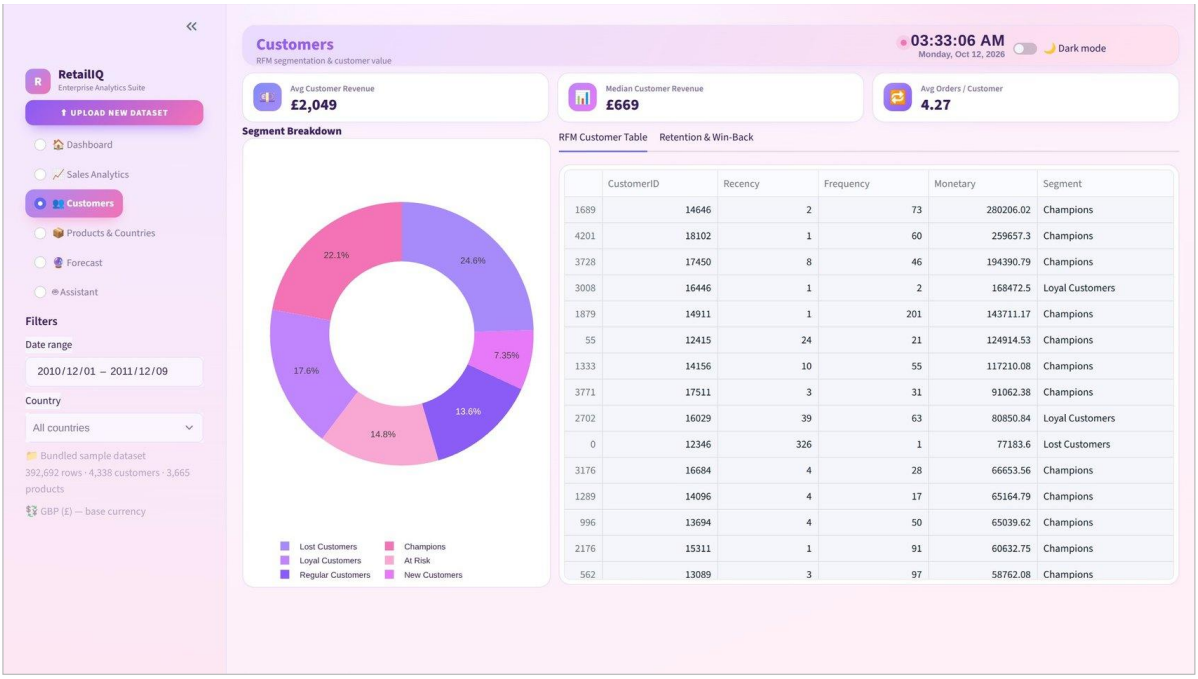


Fig No. 7.3 Customers page: segment breakdown and RFM customer table

Products and countries

User view: a gallery shows the six best-selling products with an icon or photo and revenue; a table lists the top ten products; a tab switches between a bar chart of the ten best countries and the full country table. *Technical view:* product and country performance are grouped aggregates of quantity, revenue, orders and customers sorted by revenue. In the sample data the United Kingdom accounts for about 82% of revenue, followed by the Netherlands, EIRE, Germany, France and Australia.

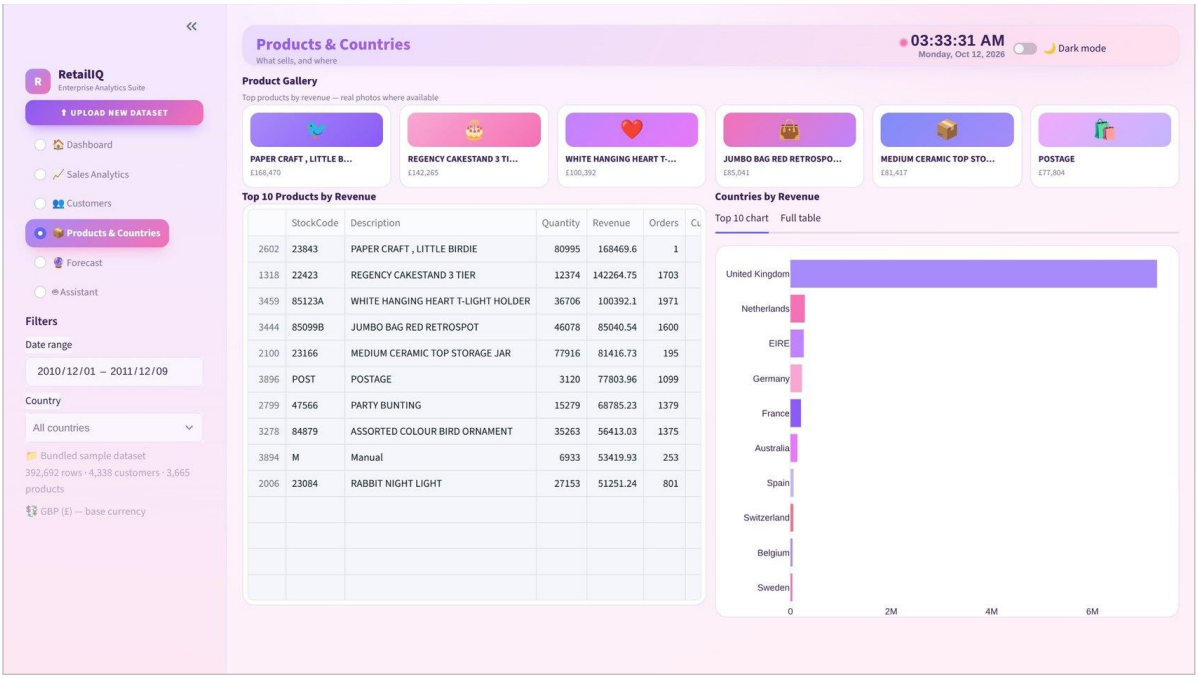


Fig No. 7.4 Products & Countries page

Revenue forecast

User view: a slider selects a horizon of one to six months; the chart shows actual revenue, the fitted trend and the projection, the table lists the forecast values and a card shows each forecast month. *Technical view:* a Linear Regression model is fitted on the filtered monthly series (at least three months are required) and extended with the future month indices.

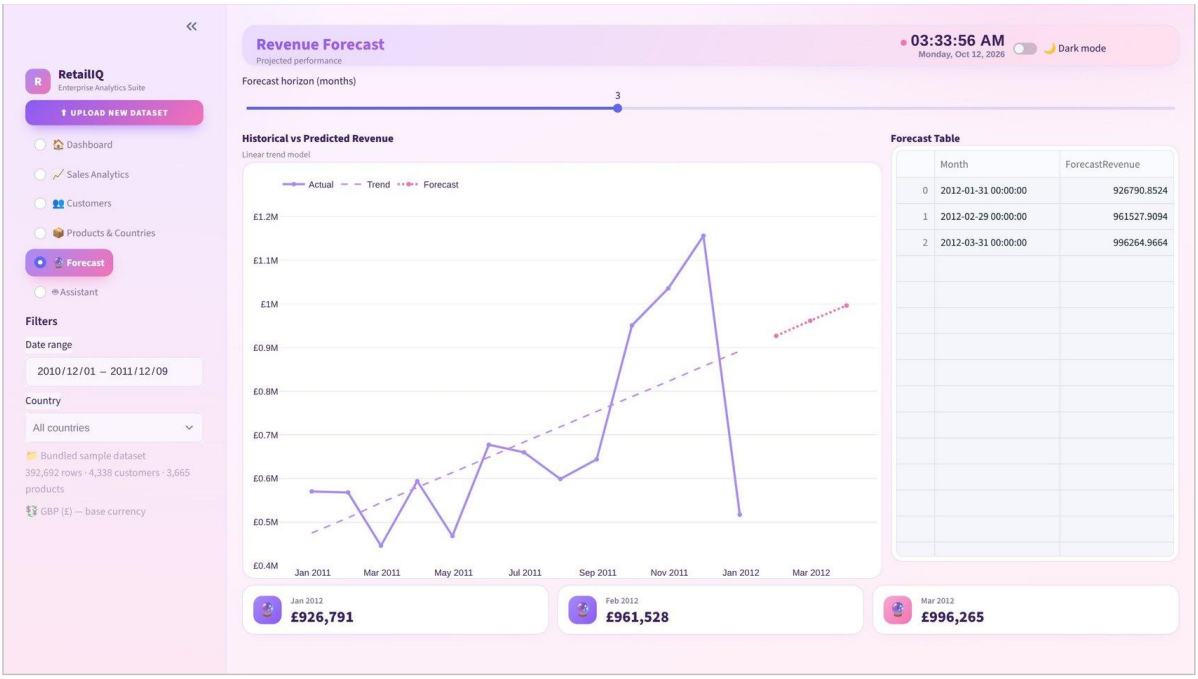


Fig No. 7.5 Forecast page: actual revenue, fitted trend and projection

AI assistant

User view: the user types a question or clicks a suggestion such as "Top 5 products" or "Customer segments" and receives an answer in the chat. A switch turns on AI mode for open-ended questions. *Technical view:* rule mode matches the question to an intent and formats an answer from the computed tables; AI mode calls the Gemini model with retry and fallback as described in Section 6.5.

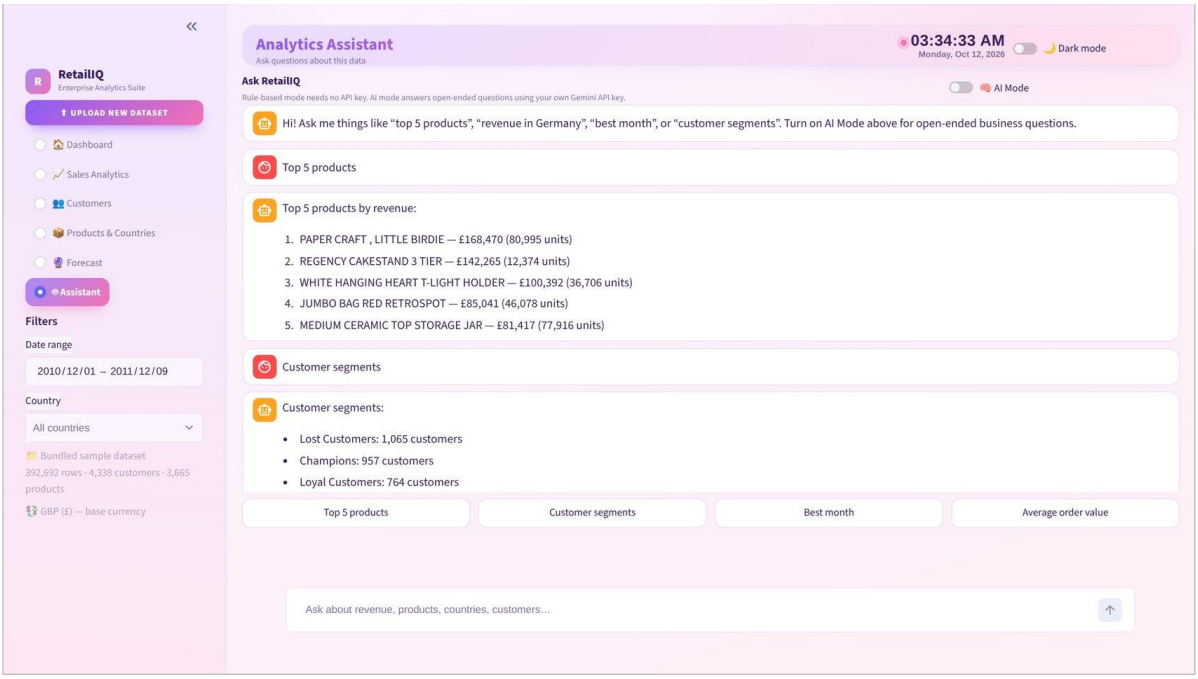


Fig No. 7.6 AI Assistant page answering questions about top products and customer segments

Filters, currency, themes and live clock

User view: the sidebar date range and country filters change every page at once. Choosing one country shows amounts in that country's currency. The top bar has a dark-mode switch and a live date and time. *Technical view:* filters create a filtered copy of the data that all analytical functions use; the stylesheet is regenerated from the theme flag; the clock is updated by a script every second.

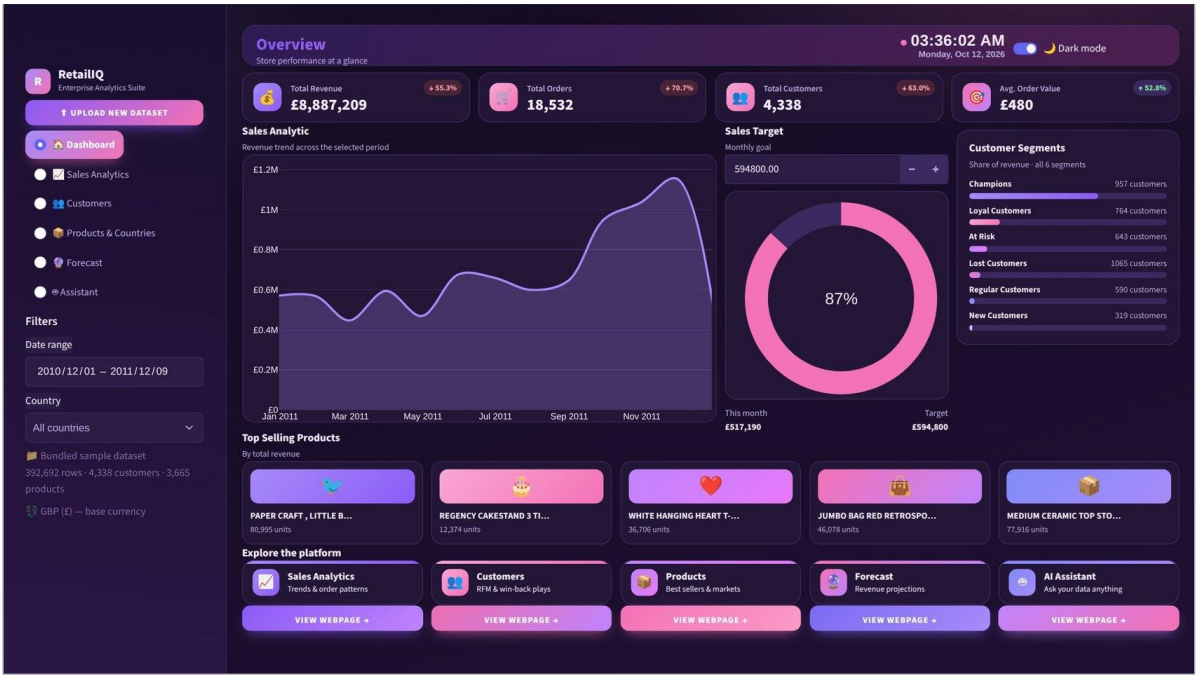


Fig No. 7.7 Overview page in dark mode

CHAPTER 8

Testing

8.1 API and backend testing (Postman)

Postman is a tool for testing REST endpoints. RetailIQ does not expose a REST API: the Streamlit server calls the Python modules directly, so there are no routes or controllers to send requests to, and Postman was therefore not used. The equivalent backend testing was done at function level. The loading and validation function was called directly with files that had the wrong columns and with a corrupted Excel file, and the cleaning, RFM, segmentation, forecasting and retention functions were exercised on the sample data (their outputs are reported in Chapters 6 and 12).

The only outgoing API call is the Gemini request. Because it cannot be called reliably in automated tests (it needs a key and depends on service load), it was tested with a simulated client that returns scripted successes and failures. The scenarios and results are given in Table 8.3.

8.2 Integration testing

End-to-end tests drive the complete application the way a user would. Streamlit's AppTest framework starts the application, clicks the sample-dataset button, opens each of the six pages in the light and in the dark theme, applies a country filter, clicks the View Webpage and Upload-new-dataset buttons and checks that no exception is raised. A headless Chromium browser was used for what AppTest cannot check: real mouse clicks (the dark-mode switch), the live clock, and the measured height of every page against the window height at 1366 x 768, 1920 x 1080 and 1366 x 650. These browser tests also found a real defect, an invisible Streamlit header that blocked clicks on the top-bar switch, which was then fixed (Section 10.1).

8.3 Tools used for testing

Table 8.1 Testing tools

Tool	Used for
Streamlit AppTest	Running the app headlessly; clicking buttons; switching pages, theme and filters; detecting exceptions
Headless Chromium (puppeteer-core)	Real browser clicks, screenshots, element positions and page-height measurements

Tool	Used for
Python mock objects	Simulating Gemini responses: success, HTTP 503, invalid key, unknown model
pandas / direct function calls	Validating cleaning counts, RFM tables, segment counts and forecast values
scikit-learn metrics	Forecast accuracy (MAE, RMSE, MAPE, R squared) in an offline backtest

8.4 Test cases and results

Table 8.2 Functional and integration test cases

ID	Test description	Expected result	Result
TC-01	Open the app and click the sample-dataset button	Dashboard opens with 4,338 customers	Pass
TC-02	Load a CSV with unrelated columns (a, b, c)	Rejected; message names missing columns	Pass
TC-03	Load a corrupted .xlsx file	Rejected with a read error, no crash	Pass
TC-04	Load a file with Online Retail II headers (Invoice, Price, Customer ID)	Columns normalised to the standard schema	Pass
TC-05	Open all six pages in the light theme	No exceptions	Pass
TC-06	Open all six pages in the dark theme	No exceptions	Pass
TC-07	Filter to Germany	Six segments listed (21, 18, 27, 10, 10, 8 customers)	Pass
TC-08	Click each View Webpage button	The matching page opens	Pass
TC-09	Click Upload new dataset	Upload page shown again	Pass
TC-10	Click the dark-mode switch with a real mouse click	Theme changes to dark	Pass (after fix)
TC-11	Page height versus window at 1366x768 and 1920x1080	All six pages fit without page scrolling	Pass
TC-12	Page height at 1366x650	Pages fit (small-window rules reduce spacing)	Pass
TC-13	Watch the clock in successive screenshots	Time advances every second	Pass

Table 8.3 Gemini retry and fallback test cases (simulated service)

ID	Scenario	Expected behaviour	Result
AI-01	503 high demand twice, then success	Answer returned on the third attempt	Pass
AI-02	Main model always busy, lite model works	Answer returned by the lighter model	Pass
AI-03	Every model busy	Clear 'service busy' message; rule-based answer shown	Pass
AI-04	Invalid or rejected API key	Reported immediately, no retries	Pass
AI-05	Unknown model name (404)	Next model in the list is tried	Pass

Forecast accuracy evaluation

The application does not show forecast errors, so an offline backtest was run for this report. The twelve complete months December 2010 to November 2011 were used (the partial December 2011 was excluded). For each of the months June to November 2011, the model was trained on all earlier months only and asked to predict that one month. Two simple baselines were evaluated in the same way: the mean of the training months, and a naive forecast equal to the previous month's revenue.

Table 8.4 One-step-ahead backtest, June to November 2011

Method	MAE (£)	RMSE (£)	MAPE (%)
Training mean	249,157	310,632	25.8
Naive (last month)	105,939	142,504	11.8
Linear Regression	150,783	189,441	15.7

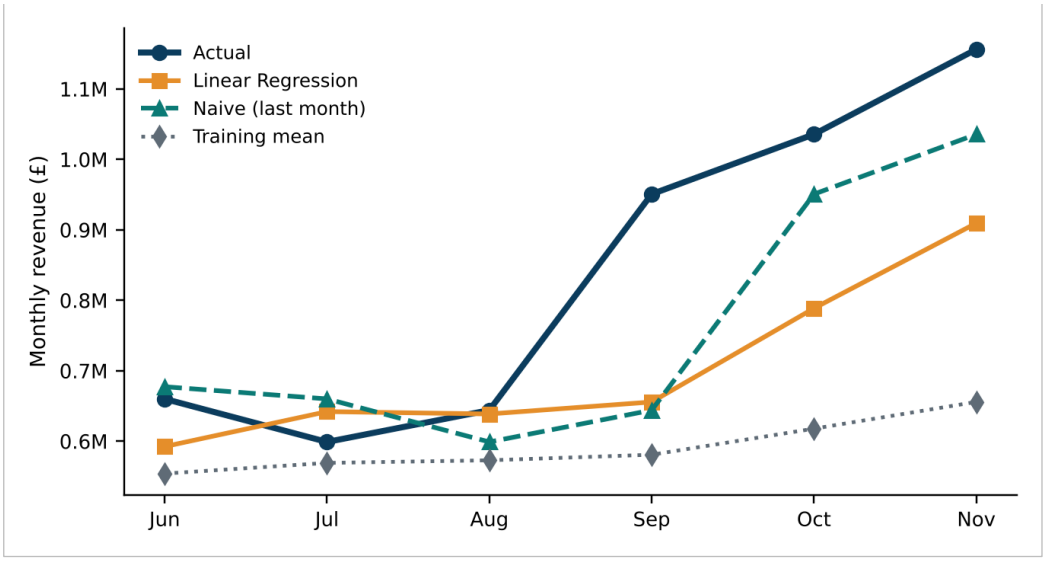


Fig No. 8.1 Backtest: actual monthly revenue versus the three methods

Fitted on all twelve complete months, the Linear Regression line explains about 68% of the variance (R squared = 0.68, in-sample MAPE 16.6%). The regression clearly beats the training mean, but in this short and fast-rising series the naive last-month forecast is more accurate, because revenue jumped sharply in September to November and a straight line trained on earlier, flatter months lags behind. This is an honest, useful finding: the model is a transparent trend baseline, and seasonality or a longer history would be needed to improve it (Chapter 11).

CHAPTER 9

Deployment

9.1 Steps to deploy

Option 1: Docker Compose (recommended)

25. Install Docker Desktop (or Docker Engine with the Compose plugin).
26. Unzip the project and open a terminal in the `RetailIQ` folder (the one that contains `Dockerfile`).
27. Optional: copy `.env.example` to `.env` and put the Gemini key after `GEMINI_API_KEY=`.
28. Run the command below. The first build downloads the Python image and installs the packages.
29. Open http://localhost:8501 in a browser.

```
docker compose up --build

# run in the background
docker compose up --build -d

# view logs / stop
docker compose logs -f
docker compose down
```

The `--build` option must be kept after every code change, otherwise Docker reuses the old image and the old interface is shown.

Option 2: Plain Docker

```
docker build -t retailiq .
docker run -p 8501:8501 --name retailiq retailiq
```

Option 3: Native Python

```
python -m venv venv
venv\Scripts\activate          # Windows  (macOS/Linux: source venv/bin/activate)
pip install -r requirements.txt
streamlit run dashboard/app.py
```

9.2 Environment configuration

Table 9.1 Configuration items

Item	Where	Purpose
GEMINI_API_KEY	.env file or environment variable	Enables AI mode; optional, rule-based mode works without it
Port 8501	Dockerfile and docker-compose.yml	Port on which Streamlit listens and the host maps to
./data volume	docker-compose.yml	Lets new datasets be added without rebuilding the image
HEALTHCHECK	Dockerfile	Calls /_stcore/health to mark the container healthy
restart: unless-stopped	docker-compose.yml	Restarts the container automatically
.streamlit/config.toml	Project folder	Base theme colours and headless server mode
requirements.txt	Project folder	Python packages (Streamlit 1.50 or newer is required)

9.3 Hosting of frontend and backend

The front end and the back end are served by the same Streamlit process, so there is only one thing to host. In the current version it is hosted locally or in a Docker container on any computer or server. If a public deployment is needed later, the same image can be pushed to a container service or virtual machine of a cloud provider, or the repository can be connected to a Streamlit hosting service. Because the application has no login system, a public deployment should be placed behind authentication.

CHAPTER 10

Challenges & Limitations

10.1 Issues faced during development

Table 10.1 Issues faced and how they were solved

Issue	Cause	Solution applied
Different dataset releases fail to load	Online Retail and Online Retail II use different column names	Alias map that renames headers; required-column check with a clear message
Charts ignored the dark theme	Streamlit fills its own light colours into Plotly figures	Set transparent backgrounds and theme font colours on every figure
Dark-mode switch could not be clicked	An invisible Streamlit header overlapped the top bar and absorbed clicks	Made the header click-through and kept its own buttons clickable
Page needed scrolling	Fixed chart heights and many stacked sections	Window-height based CSS sizing, compact components and a short-window mode
Scaling the whole page squashed charts	CSS zoom made Plotly measure its container twice	Dropped zoom; Plotly now follows CSS heights when no fixed height is set
Card HTML shown as code text	Indented HTML is treated as a code block by Markdown	Component HTML is flattened to a single line before display
Segment card listed only four segments	The list was limited to the four largest by revenue	Show all six segments; shares now add up to 100%
AI mode failed with HTTP 503	Gemini model reported high demand	Retry with back-off, fall back to lighter models and to the rule-based answer
Very slow first load of the Excel file	Reading 541,909 rows from XLSX is slow	Cached loading; bundled pre-cleaned CSV for the sample

10.2 Solutions applied

The solutions in Table 10.1 share two principles that guided the project. First, **fail clearly**: bad input produces a readable message and keeps the user on the upload page, and the AI feature degrades to a free answer instead of an error. Second, **verify visually**: the interface was checked in a real browser at several window sizes, because several defects (the blocked switch, the squashed charts, the clipped headings) were only visible there.

10.3 Current limitations or known bugs

30. **Forecast model.** A linear trend does not model seasonality, and in the backtest it is less accurate than the naive last-month forecast on this short series. The dashboard does not yet show forecast error.

31. **Partial last month.** The data end on 9 December 2011, so the last month is incomplete. It lowers the final observed point, pulls the fitted trend down and makes that month's month-over-month badge misleading.
32. **Non-merchandise stock codes.** Items such as postage (POST) and manual adjustments (M) appear in the product ranking although they are not real products.
33. **One dominant market.** The United Kingdom contributes about 82% of revenue, so country comparisons are highly imbalanced. Exchange rates used for country currencies are fixed and indicative, not live.
34. **Dataset scope.** Only online-retail transaction files with the required columns are supported; the application rejects other data.
35. **Assistant limits.** Rule-based mode covers fixed intents only; AI mode needs a Gemini key, internet access and a service that is not overloaded.
36. **No user accounts.** The application has no authentication and keeps no data between sessions.
37. **Single-file processing.** Very large files (millions of rows) will take longer and need more memory than the recommended machine.

CHAPTER 11

Future Enhancements

11.1 Planned features

38. **Customer churn prediction** with supervised learning (for example logistic regression or gradient boosting) using RFM and behavioural features.
39. **Customer lifetime value** estimation to rank customers by expected future worth, not only past spend.
40. **Better forecasting:** ARIMA or Prophet with seasonality, XGBoost or LSTM models, a walk-forward evaluation shown in the dashboard, and confidence bands around the forecast.
41. **Anomaly detection** for unusual orders, returns and price entries.
42. **Product analytics:** removal of non-merchandise codes, market-basket analysis and product recommendations.
43. **Export features:** download of segments, recommendations and charts as CSV, Excel and PDF reports.
44. **Multi-language and multi-currency** support with live exchange rates.

11.2 Possible integrations or optimizations

45. Connect directly to e-commerce platforms or to a relational database so that data need not be uploaded manually.
46. Add user accounts, roles and saved workspaces, and deploy to a cloud service with a CI/CD pipeline.
47. Speed up the first load of large files with chunked reading, Parquet storage or a database back end.
48. Send retention recommendations automatically to e-mail or messaging tools as campaigns.
49. Add explainable-AI views so that users can see why a customer was placed in a segment or why the forecast changed.

CHAPTER 12

Conclusion

12.1 Summary of the project

RetailIQ is an end-to-end analytics application for online-retail transaction data. A user uploads a file; the system normalises, validates and cleans it, derives revenue and time features, scores and segments the customers, ranks products and countries, projects revenue and presents everything in an interactive dashboard with a conversational assistant. It is packaged with Docker so that it runs the same way anywhere.

12.2 What was achieved

Table 12.1 Objectives and outcomes

Objective	Outcome
1. Reliable cleaning pipeline	541,909 raw records reduced to 392,692 clean records; 5,268 duplicates, 135,037 lines without customer, 8,872 cancelled lines and 40 invalid prices removed; files with other headers are normalised or rejected clearly. Achieved.
2. RFM segmentation	4,338 customers placed in six segments; Champions are 22.1% of customers and 65.2% of revenue. Achieved.
3. Retention recommendations	Every segment has a headline and 3 to 5 concrete actions, with customers and revenue at stake shown per segment. Achieved.
4. Forecasting with honest accuracy	Linear Regression forecasts 1 to 6 months (for example £926,791 for January 2012); backtest MAPE 15.7%, versus 11.8% for a naive baseline and 25.8% for the mean. Achieved, with the limitations stated.
5. Upload-first dashboard	Six pages, filters, light and dark themes, live clock and a fixed no-scroll layout, tested at three window sizes. Achieved.
6. Assistant and Docker	Rule-based and Gemini-based assistant with retry and fallback; Dockerfile and Compose file provided. Achieved.

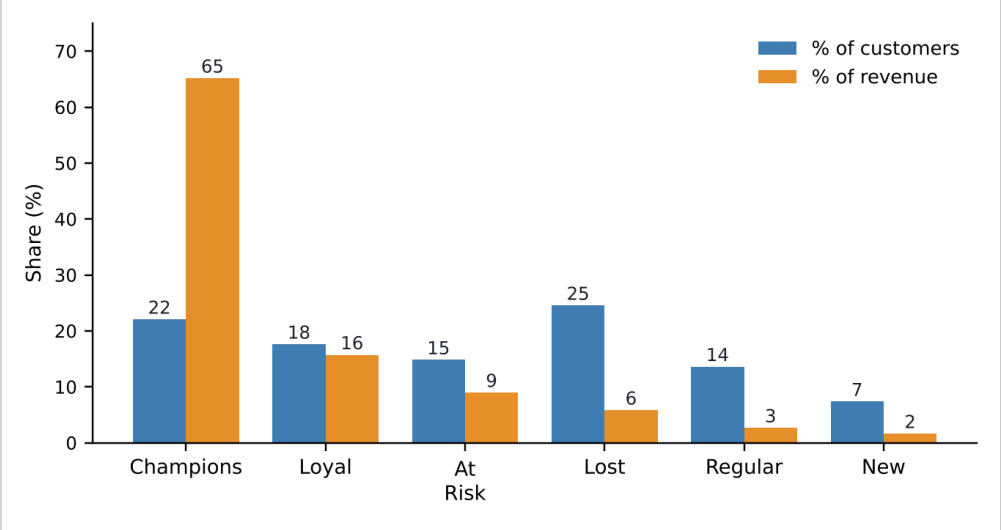


Fig No. 12.1 Share of customers and share of revenue by segment

12.3 Skills learned during development

50. Data cleaning and validation of real-world transaction data with pandas, and the importance of measuring what every rule removes.
51. Customer analytics: RFM scoring, rule-based segmentation and turning segments into business actions.
52. Time-series basics: monthly aggregation, linear trend fitting, and honest evaluation with a backtest and baselines.
53. Building interactive applications with Streamlit and Plotly, including state management, caching and callbacks.
54. Front-end craft: CSS variables, light and dark theming, responsive sizing and visual testing in a real browser.
55. Integrating a generative-AI service robustly: prompt grounding, error classification, retry with back-off and graceful fallback.
56. Software engineering practice: modular code, automated and simulated testing, containerisation with Docker, documentation and teamwork.

CHAPTER 13

References

The following books, research articles, datasets and documentation were used in this project:

[1] D. Chen, S. L. Sain and K. Guo, "Data mining for the online retail industry: A case study of RFM model-based customer segmentation using data mining," Journal of Database Marketing & Customer Strategy Management, 2012.

[2] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[3] A. M. Hughes, Strategic Database Marketing. Chicago, IL, USA: Probus Publishing, 1994.

[4] UCI Machine Learning Repository, "Online Retail," Dataset 352, University of California, Irvine. [Online]. Available: https://archive.ics.uci.edu/dataset/352/online+retail

[5] UCI Machine Learning Repository, "Online Retail II," Dataset 502, University of California, Irvine. [Online]. Available: https://archive.ics.uci.edu/dataset/502/online+retail+ii

[6] Streamlit Inc., "Streamlit documentation." [Online]. Available: https://docs.streamlit.io

[7] Docker Inc., "Docker documentation." [Online]. Available: https://docs.docker.com

[8] Google, "Gemini API documentation," Google AI for Developers. [Online]. Available: https://ai.google.dev/gemini-api/docs

[9] W. McKinney, "Data structures for statistical computing in Python," in Proc. 9th Python in Science Conf., 2010, pp. 56-61.

[10] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, pp. 357-362, 2020.

[11] Plotly Technologies Inc., "Collaborative data science," Montreal, QC, 2015. [Online]. Available: https://plotly.com

[12] Scikit-learn Developers, "LinearRegression," Scikit-learn documentation. [Online]. Available: https://scikit-learn.org

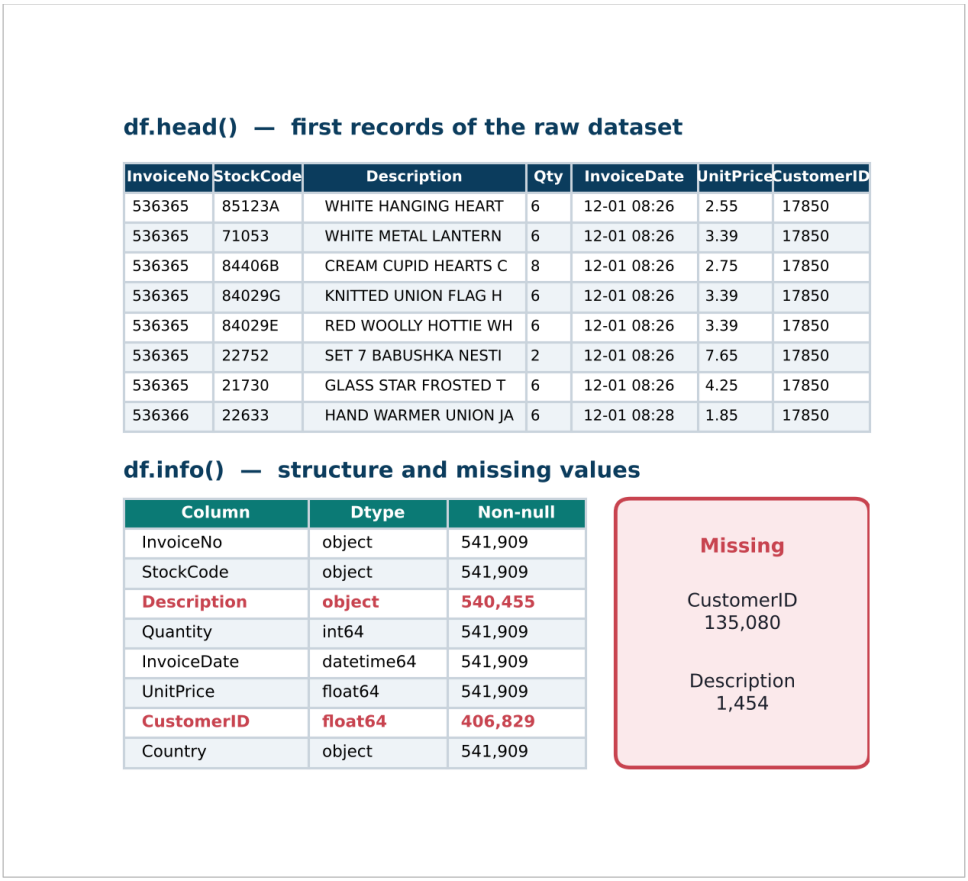
[13] J. D. Hunter, "Matplotlib: A 2D graphics environment," Computing in Science & Engineering, vol. 9, no. 3, pp. 90-95, 2007.

CHAPTER 14

Appendices

A. Screenshots of the app

The screenshots of the individual pages are included in Section 7.2. Fig. 14.1 shows the cleaned-data profile that was used to understand the dataset before modelling.



df.head() — first records of the raw dataset

InvoiceNo	StockCode	Description	Qty	InvoiceDate	UnitPrice	CustomerID
536365	85123A	WHITE HANGING HEART	6	12-01 08:26	2.55	17850
536365	71053	WHITE METAL LANTERN	6	12-01 08:26	3.39	17850
536365	84406B	CREAM CUPID HEARTS C	8	12-01 08:26	2.75	17850
536365	84029G	KNITTED UNION FLAG H	6	12-01 08:26	3.39	17850
536365	84029E	RED WOOLLY HOTTIE WH	6	12-01 08:26	3.39	17850
536365	22752	SET 7 BABUSHKA NESTI	2	12-01 08:26	7.65	17850
536365	21730	GLASS STAR FROSTED T	6	12-01 08:26	4.25	17850
536366	22633	HAND WARMER UNION JA	6	12-01 08:28	1.85	17850

df.info() — structure and missing values

Column	Dtype	Non-null
InvoiceNo	object	541,909
StockCode	object	541,909
Description	**object**	**540,455**
Quantity	int64	541,909
InvoiceDate	datetime64	541,909
UnitPrice	float64	541,909
CustomerID	**float64**	**406,829**
Country	object	541,909

Missing

CustomerID
135,080

Description
1,454

Fig No. 14.1 Profile of the raw dataset: first records, column types and missing values

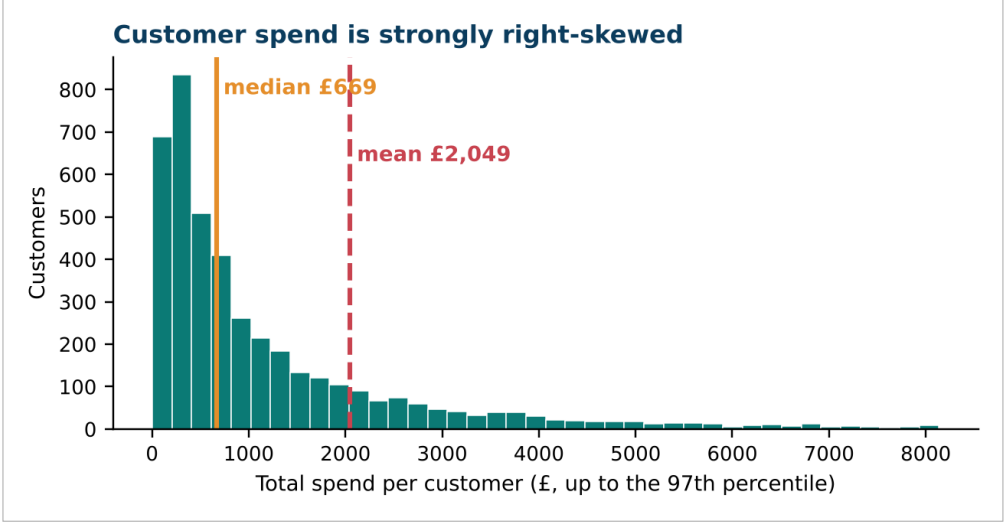


Fig No. 14.2 Distribution of customer spend (right-skewed; mean far above median)

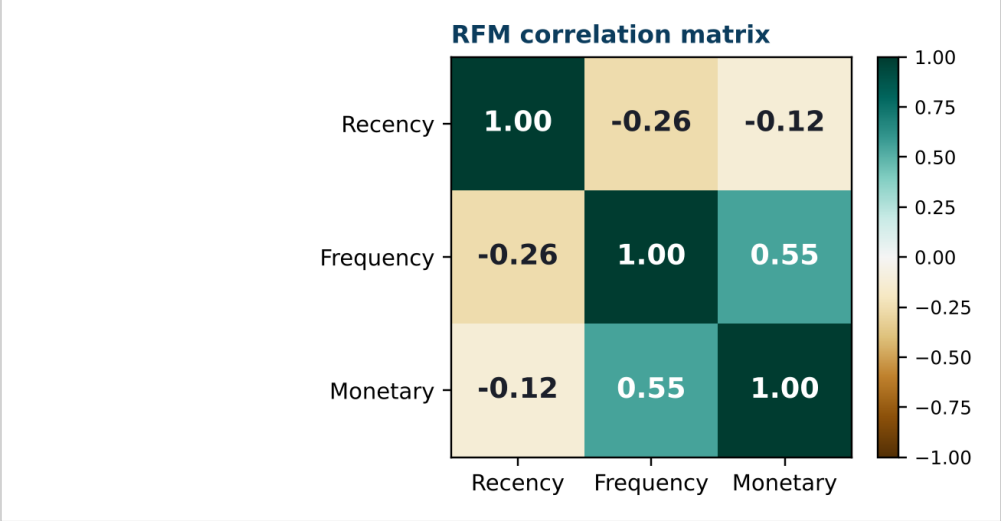


Fig No. 14.3 Correlation between Recency, Frequency and Monetary values

B. Sample code snippets

B.1 Column normalisation and cleaning (src/data_cleaning.py)

```
def normalize_columns(df):
    """
    Rename whatever headers the source file uses to RetailIQ's standard
    schema, so Online Retail I, Online Retail II, or any similarly-shaped
    export can all be fed through the same pipeline.
    """

    df = df.copy()

    rename_map = {}
    for col in df.columns:
        key = str(col).strip().lower()
        if key in COLUMN_ALIASES:
            rename_map[col] = COLUMN_ALIASES[key]

    df = df.rename(columns=rename_map)
```

```
    missing = [c for c in REQUIRED_COLUMNS if c not in df.columns]
    if missing:
        raise ValueError(
            "This file is missing column(s) RetailIQ needs: "
            f"{', '.join(missing)}. Expected a transactional export with "
            "invoice, product, quantity, price, customer and country "
            "columns (like the UCI 'Online Retail' dataset)."
        )

    return df[REQUIRED_COLUMNS]

def clean_data(df):
    """Clean the retail transaction dataset."""

    df = df.copy()

    # Remove duplicate rows
    df = df.drop_duplicates()

    # Remove rows without CustomerID
    df = df.dropna(subset=["CustomerID"])

    # Remove returns / negative quantities
    df = df[df["Quantity"] > 0]

    # Remove zero or negative prices
    df = df[df["UnitPrice"] > 0]

    return df
```

B.2 RFM table (src/customer_analysis.py)

```
def calculate_rfm(df, analysis_date=None):
    """Calculate Recency, Frequency and Monetary values."""

    df = df.copy()

    df["InvoiceDate"] = pd.to_datetime(df["InvoiceDate"])

    if analysis_date is None:
        analysis_date = df["InvoiceDate"].max() + pd.Timedelta(days=1)

    rfm = (
        df.groupby("CustomerID")
        .agg(
            Recency=("InvoiceDate",
                     lambda x: (analysis_date - x.max()).days),
            Frequency=("InvoiceNo", "nunique"),
            Monetary=("Revenue", "sum")
        )
        .reset_index()
    )

    return rfm
```

B.3 RFM scores and segment rules (src/segmentation.py)

```
def calculate_rfm_scores(rfm):
    """Create RFM scores from Recency, Frequency and Monetary values."""

    rfm = rfm.copy()
```

```
    rfm["R_Score"] = pd.qcut(
        rfm["Recency"],
        5,
        labels=[5, 4, 3, 2, 1],
        duplicates="drop"
    ).astype(int)

    rfm["F_Score"] = pd.qcut(
        rfm["Frequency"].rank(method="first"),
        5,
        labels=[1, 2, 3, 4, 5]
    ).astype(int)

    rfm["M_Score"] = pd.qcut(
        rfm["Monetary"].rank(method="first"),
        5,
        labels=[1, 2, 3, 4, 5]
    ).astype(int)

    rfm["RFM_Score"] = (
        rfm["R_Score"].astype(str)
        + rfm["F_Score"].astype(str)
        + rfm["M_Score"].astype(str)
    )

    return rfm

def assign_customer_segments(rfm):
    """Assign customers to business-oriented segments."""

    rfm = rfm.copy()

    def segment_customer(row):

        r = row["R_Score"]
        f = row["F_Score"]
        m = row["M_Score"]

        if r >= 4 and f >= 4 and m >= 4:
            return "Champions"

        elif r >= 3 and f >= 3 and m >= 3:
            return "Loyal Customers"

        elif r >= 4 and f <= 2:
            return "New Customers"

        elif r <= 2 and f >= 3:
            return "At Risk"

        elif r <= 2 and f <= 2:
            return "Lost Customers"

        else:
            return "Regular Customers"

    rfm["Segment"] = rfm.apply(segment_customer, axis=1)

    return rfm
```

B.4 Forecasting (src/forecasting.py)

```
def create_monthly_sales(df):
```

```
    """Create monthly revenue dataset."""

    df = df.copy()

    df["InvoiceDate"] = pd.to_datetime(df["InvoiceDate"])

    monthly_sales = (
        df.set_index("InvoiceDate")
        .resample("ME")["Revenue"]
        .sum()
        .reset_index()
    )

    monthly_sales.columns = ["Month", "Revenue"]

    return monthly_sales

def train_forecasting_model(monthly_sales):
    """Train a baseline linear regression forecasting model."""

    data = monthly_sales.copy()

    data["TimeIndex"] = np.arange(len(data))

    X = data[["TimeIndex"]]
    y = data["Revenue"]

    model = LinearRegression()

    model.fit(X, y)

    data["PredictedRevenue"] = model.predict(X)

    return model, data

def forecast_future(model, monthly_sales, periods=3):
    """Forecast future monthly revenue."""

    last_index = len(monthly_sales) - 1

    future_indices = np.arange(
        last_index + 1,
        last_index + periods + 1
    )

    predictions = model.predict(
        future_indices.reshape(-1, 1)
    )

    future_dates = pd.date_range(
        start=monthly_sales["Month"].max() + pd.offsets.MonthEnd(1),
        periods=periods,
        freq="ME"
    )

    forecast = pd.DataFrame({
        "Month": future_dates,
        "ForecastRevenue": predictions
    })

    return forecast
```

B.5 Gemini call with retry and fallback (src/ai_assistant.py)

```
def _is_temporary(err: str) -> bool:
    """True for errors that usually go away by themselves (overload, rate limit,
timeout)."""
    e = err.lower()
    return any(k in e for k in ("503", "unavailable", "overloaded", "high demand",
                                "429", "resource_exhausted", "500", "internal",
                                "504", "deadline", "timeout", "timed out"))

def _is_auth_error(err: str) -> bool:
    e = err.lower()
    return any(k in e for k in ("api key", "api_key", "permission_denied", "401",
"403", "unauthenticated"))

def ask_ai(api_key: str, question: str, data_context: str, chat_history=None, model:
str = DEFAULT_MODEL):
    """
    Ask the Gemini API a question grounded in the given data context.

    Returns (answer_text, error_text). Exactly one of the two is None.
    Temporary errors (503 high demand, 429, timeouts) are retried with a short
    back-off, then the next model in FALLBACK_MODELS is tried.
    """
    if not _GENAI_AVAILABLE:
        return None, (
            "The `google-genai` package isn't installed. Run "
            "`pip install google-genai` and restart the app to enable AI mode."
        )
    if not api_key:
        return None, "No Gemini API key configured. Add one in the sidebar or set
GEMINI_API_KEY."

    history_text = ""
    if chat_history:
        # Keep only the last few turns so the prompt stays small/cheap.
        recent = chat_history[-6:]
        history_text = "\n".join(f"{m['role'].upper()}: {m['content']}" for m in
recent)

    prompt = (
        f"{SYSTEM_INSTRUCTION}\n\n"
        f"=== DATA SUMMARY ===\n{data_context}\n\n"
        f"=== RECENT CONVERSATION ===\n{history_text}\n\n"
        f"=== QUESTION ===\n{question}"
    )

    try:
        client = genai.Client(api_key=api_key)
    except Exception as e:  # noqa: BLE001
        return None, f"Couldn't connect to the Gemini API: {e}"

    models = [model] + [m for m in FALLBACK_MODELS if m != model]
    last_error, saw_temporary = None, False

    for attempt_model in models:
        for attempt in range(RETRIES_PER_MODEL):
            try:
                response = client.models.generate_content(model=attempt_model,
contents=prompt)
                text = getattr(response, "text", None)
                if text:
                    return text.strip(), None
                last_error = "The AI returned an empty response. Try rephrasing the
question."
                break  # try the next model
            except Exception as e:  # noqa: BLE001
```

```
                last_error = str(e)
                if _is_auth_error(last_error):
                    return None, "The Gemini API key was rejected. Check that the key
is correct and active."
                if _is_temporary(last_error):
                    saw_temporary = True
                    if attempt < RETRIES_PER_MODEL - 1:
                        time.sleep(BACKOFF_SECONDS[min(attempt, len(BACKOFF_SECONDS) -
1)])
                        continue
                break  # not temporary (e.g. unknown model) or retries used up: next
model

    if saw_temporary:
        return None, (
            "Gemini is very busy right now (high demand). I retried several times and
"
            "also tried other models. Please try again in a minute."
        )
    return None, f"AI request failed: {last_error}"
```

C. Installation/setup instructions

1. Install Python 3.11 (or Docker Desktop).
2. Unzip the project folder RetailIQ.
3. Docker: run docker compose up --build in the folder and open http://localhost:8501.
4. Without Docker: create a virtual environment, run pip install -r requirements.txt and then streamlit run dashboard/app.py.
5. Optional AI mode: copy .env.example to .env and set GEMINI_API_KEY.

D. User manual or guide

1. **Start.** Open the application. The upload page appears.
2. **Load data.** Drag a CSV or Excel online-retail file onto the box, or click *Try with the sample dataset*. Wait for the analysis panel to finish.
3. **Explore.** Use the sidebar to open Dashboard, Sales Analytics, Customers, Products & Countries, Forecast and Assistant, or click a View Webpage button on the Overview page.
4. **Filter.** Choose a date range and, if wanted, one or more countries. Every page updates.
5. **Understand customers.** On the Customers page open the *Retention & Win-Back* tab and select a segment to read the recommended actions.
6. **Forecast.** On the Forecast page choose one to six months with the slider.

7. **Ask questions.** On the Assistant page type a question or click a suggestion. Switch on *AI Mode* for open-ended questions (a Gemini key is needed).
8. **Change theme.** Use the *Dark mode* switch in the top bar.
9. **New file.** Click *Upload new dataset* at the top of the sidebar.

E. Team Contributions

Kolapally Vasavi:

- **Developed the data and analytics pipeline.**
- **Worked on upload validation and schema normalisation.**
- **Implemented data cleaning and preprocessing.**
- **Worked on feature engineering.**
- **Implemented RFM scoring and customer segmentation.**
- **Developed the customer retention playbook.**
- **Worked on creating and maintaining notebooks.**

Kunchala Naga Chaitanya Sphoorthy:

- **Worked on analytics, forecasting and assistant development.**
- **Performed product-wise and country-wise analysis.**
- **Implemented currency conversion functionality.**
- **Developed revenue forecasting and backtesting.**
- **Implemented a rule-based chatbot.**
- **Integrated the Gemini assistant with retry and fallback mechanisms.**
- **Worked on testing and validating the analytics and assistant functionalities.**

M. Haswanthi:

- **Developed the dashboard and deployment functionalities.**
- **Created interactive Streamlit pages.**
- **Implemented the file upload flow.**
- **Worked on light and dark theme functionality.**
- **Implemented a live clock feature.**
- **Worked on fixing and maintaining the dashboard layout.**
- **Handled application deployment using Docker and Docker Compose.**

All three team members contributed to project integration, testing, debugging and documentation according to their respective responsibilities.

F. Certifications

https://github.com/sphoorthy282007/DATA-SCIENCE_510/tree/main/Project/RetailIQ